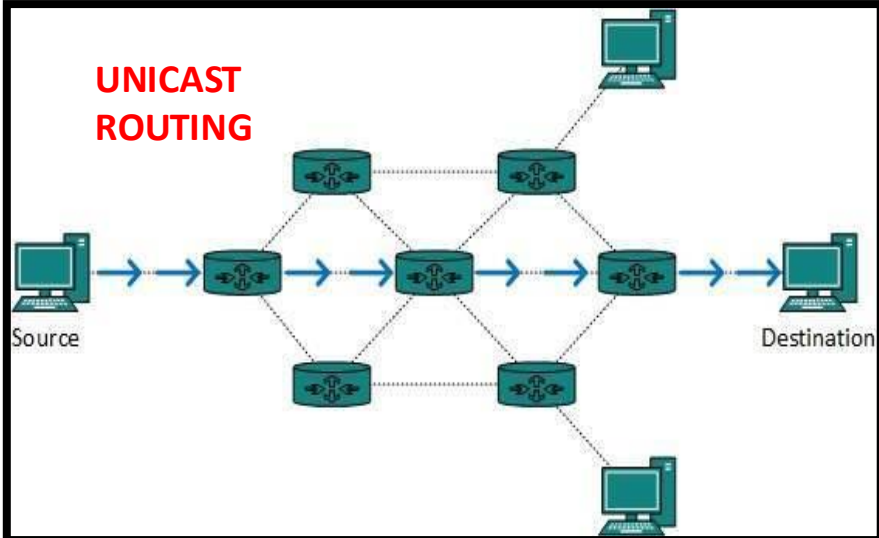


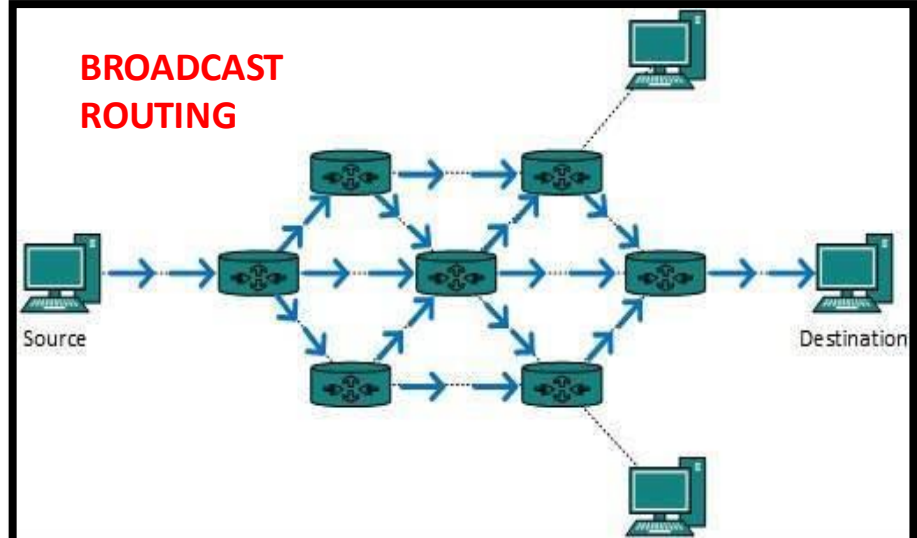
Routing Algorithms

Routes can be statically configured or dynamically learnt.

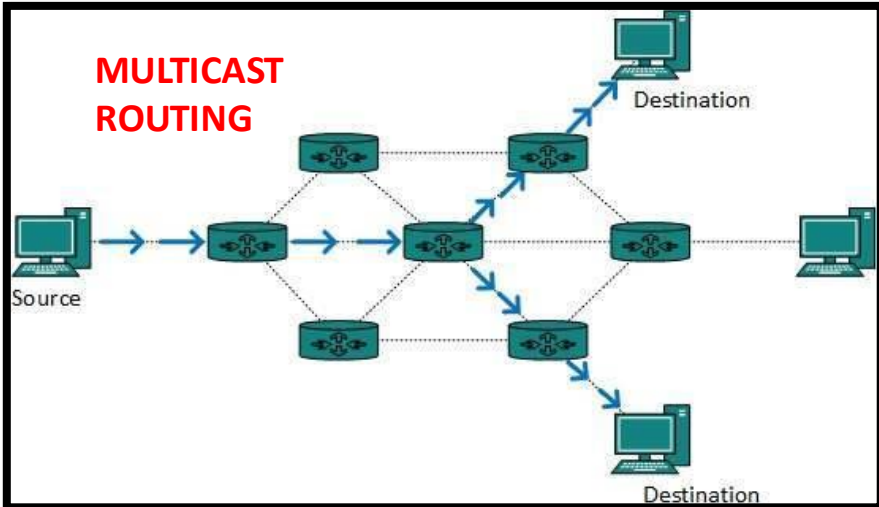
UNICAST ROUTING



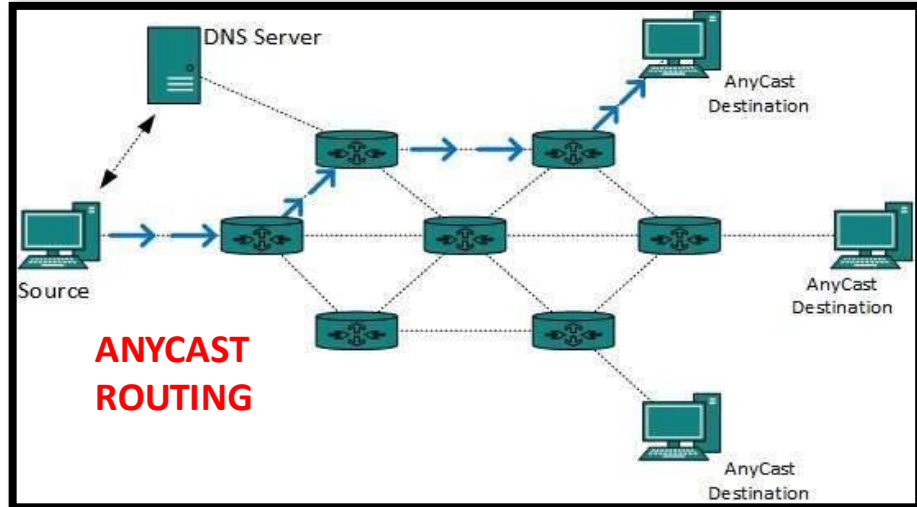
BROADCAST ROUTING

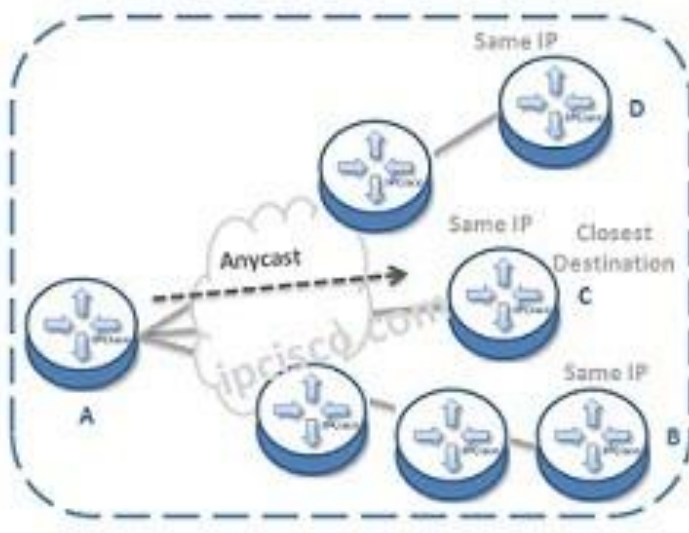
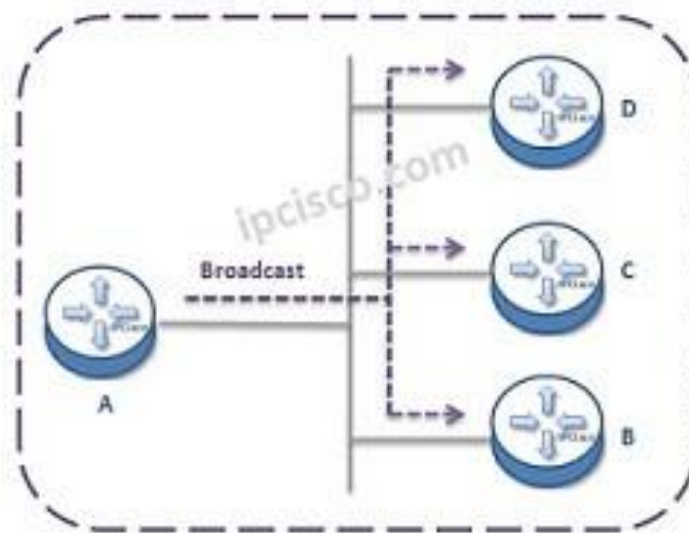
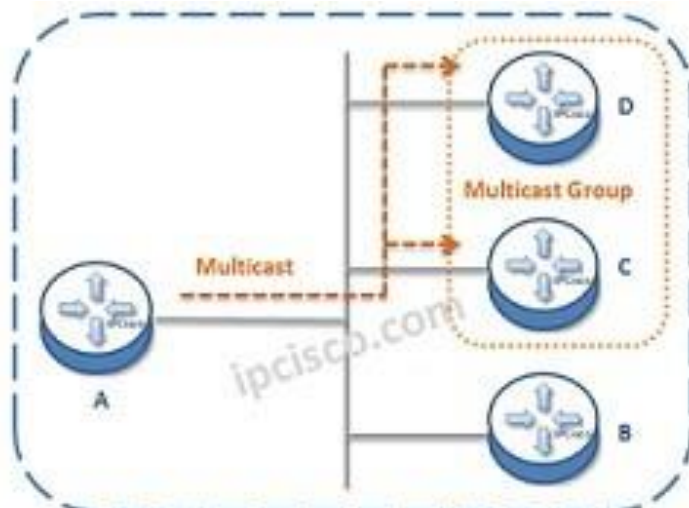
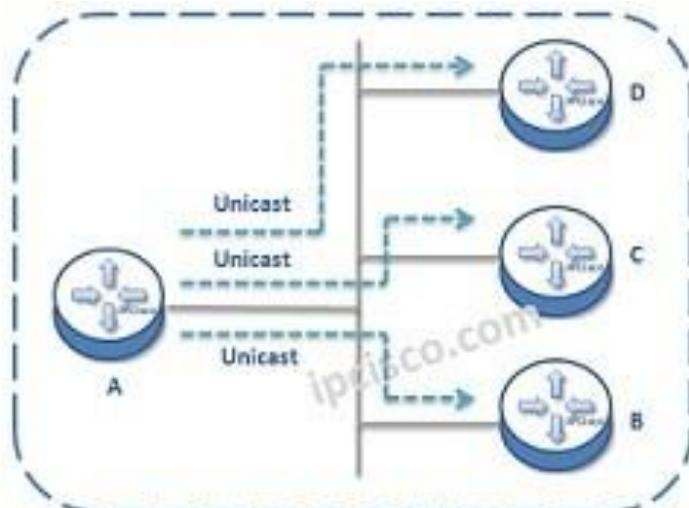


MULTICAST ROUTING

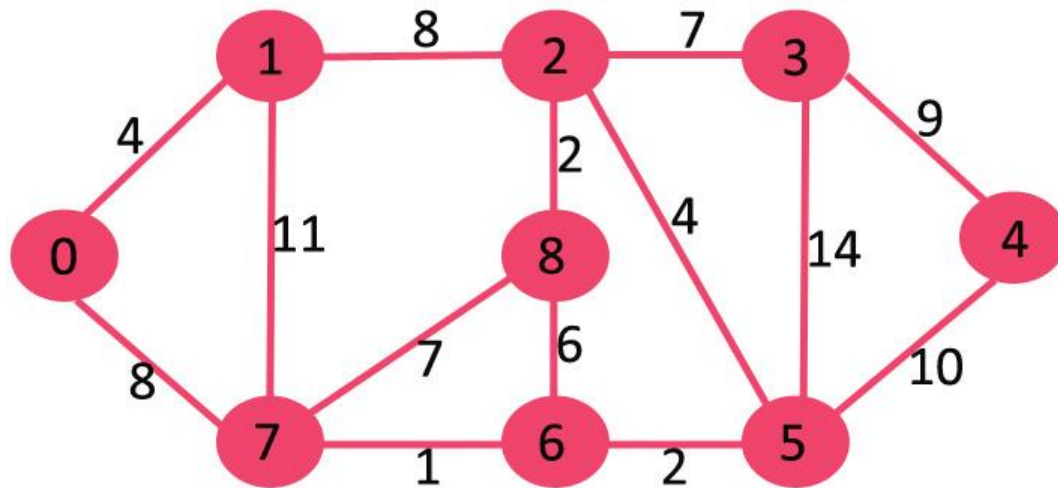


ANYCAST ROUTING



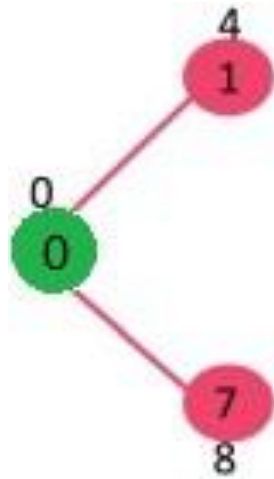


Dijkstra's Shortest Path Algorithm



*The set **sptSet** is initially empty.*

Distances assigned from Source '0' to vertices = $\{0, \infty, \infty, \infty, \infty, \infty, \infty, \infty\}$.

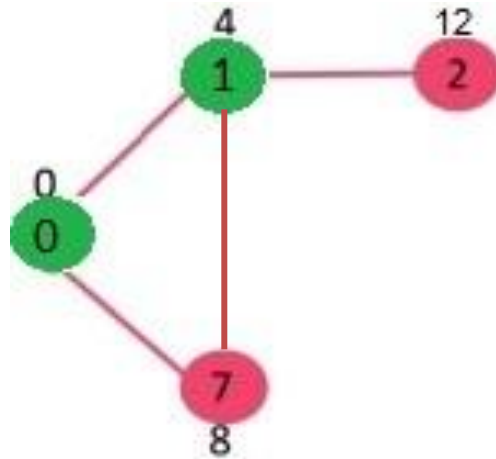


Pick the vertex with a minimum distance value.

Vertex 0 is picked.

$sptSet = \{0\}$

Distance values of 1 and 7 = updated as 4 and 8, respectively.

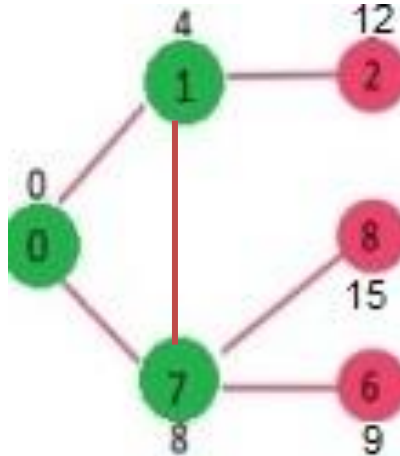


Pick the vertex with minimum distance value & not already included in sptSET.

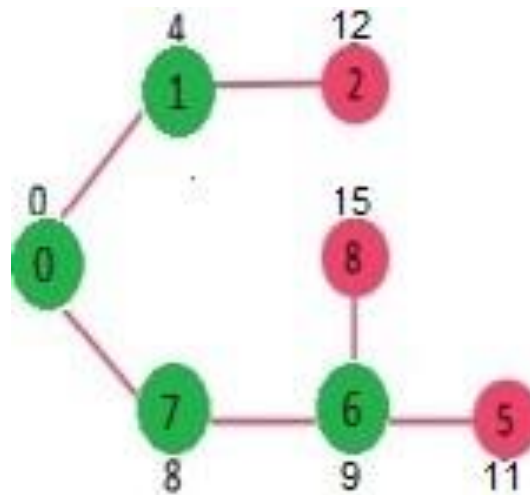
Vertex 1 is picked.

sptSet = {0, 1}

Distance value of 2 = updated as 12.



*Pick the vertex with minimum distance value & not already included in sptSET.
Vertex 7 is picked.
 $sptSet = \{0, 1, 7\}$
Distance values of 6 and 8 = updated as 15 and 9, respectively.*

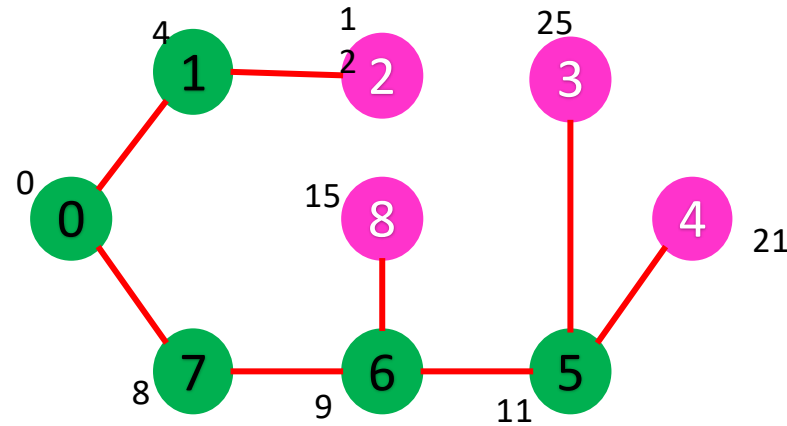


*Pick the vertex with minimum distance value & not already included in *sptSET*.*

Vertex 6 is picked.

$sptSet = \{0, 1, 7, 6\}$

Distance values of 5 and 8 = updated as 11 and 15, respectively.

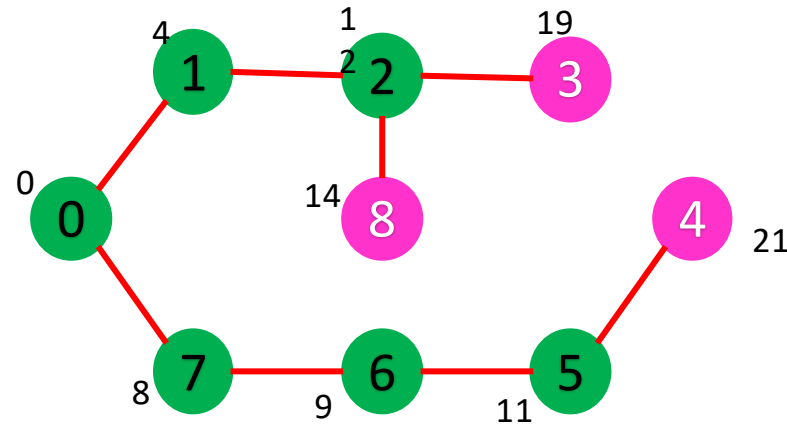


Pick the vertex with minimum distance value & not already included in sptSET.

Vertex 5 is picked.

sptSet = {0, 1, 7, 6, 5}

Distance values of 3 and 4 = updated as 25 and 21, respectively.

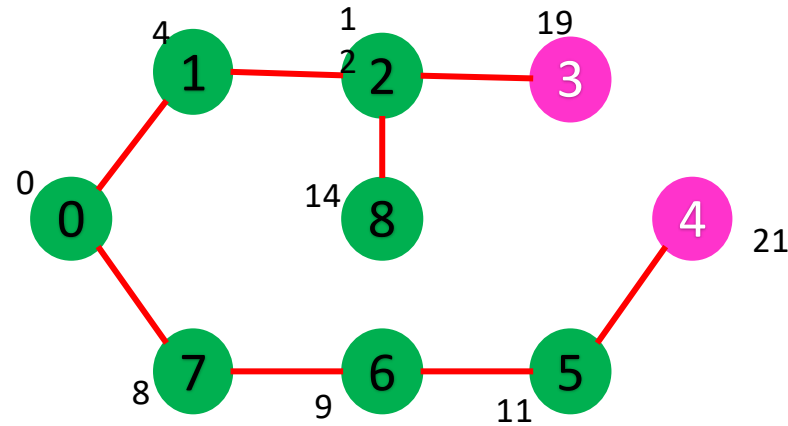


Pick the vertex with minimum distance value & not already included in sptSET.

Vertex 2 is picked.

sptSet = {0, 1, 7, 6, 5, 2}

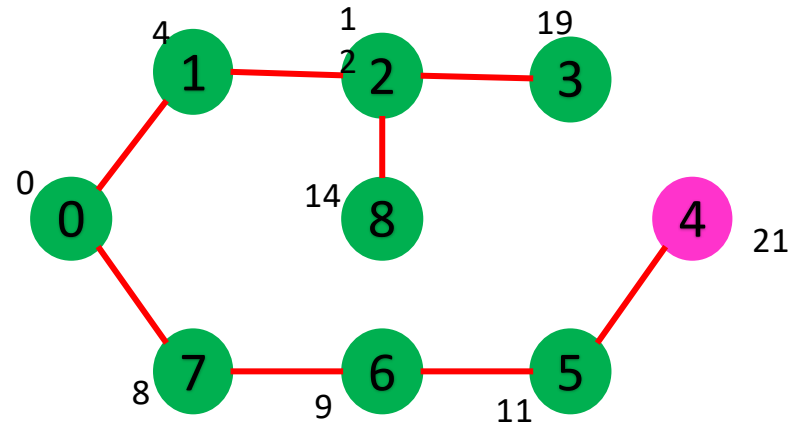
Distance values of 3 and 8 = updated as 19 and 14, respectively.



Pick the vertex with minimum distance value & not already included in sptSET.

Vertex 8 is picked.

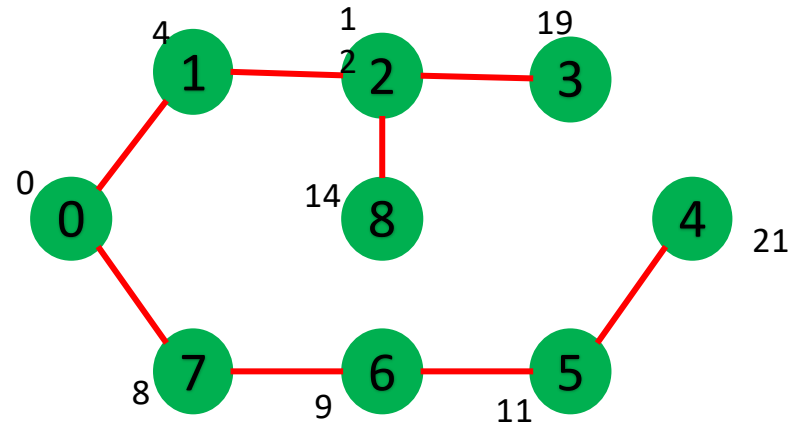
sptSet = {0, 1, 7, 6, 5, 2, 8}



Pick the vertex with minimum distance value & not already included in sptSET.

Vertex 3 is picked.

sptSet = {0, 1, 7, 6, 5, 2, 8, 3}



Pick the vertex with minimum distance value & not already included in $sptSET$.

Vertex 4 is picked.

$sptSet = \{0, 1, 7, 6, 5, 2, 8, 3, 4\}$

Distance Vector Routing Protocol

Distance Vector is simple routing protocol which takes **routing decision on the number of hops between source and destination.**

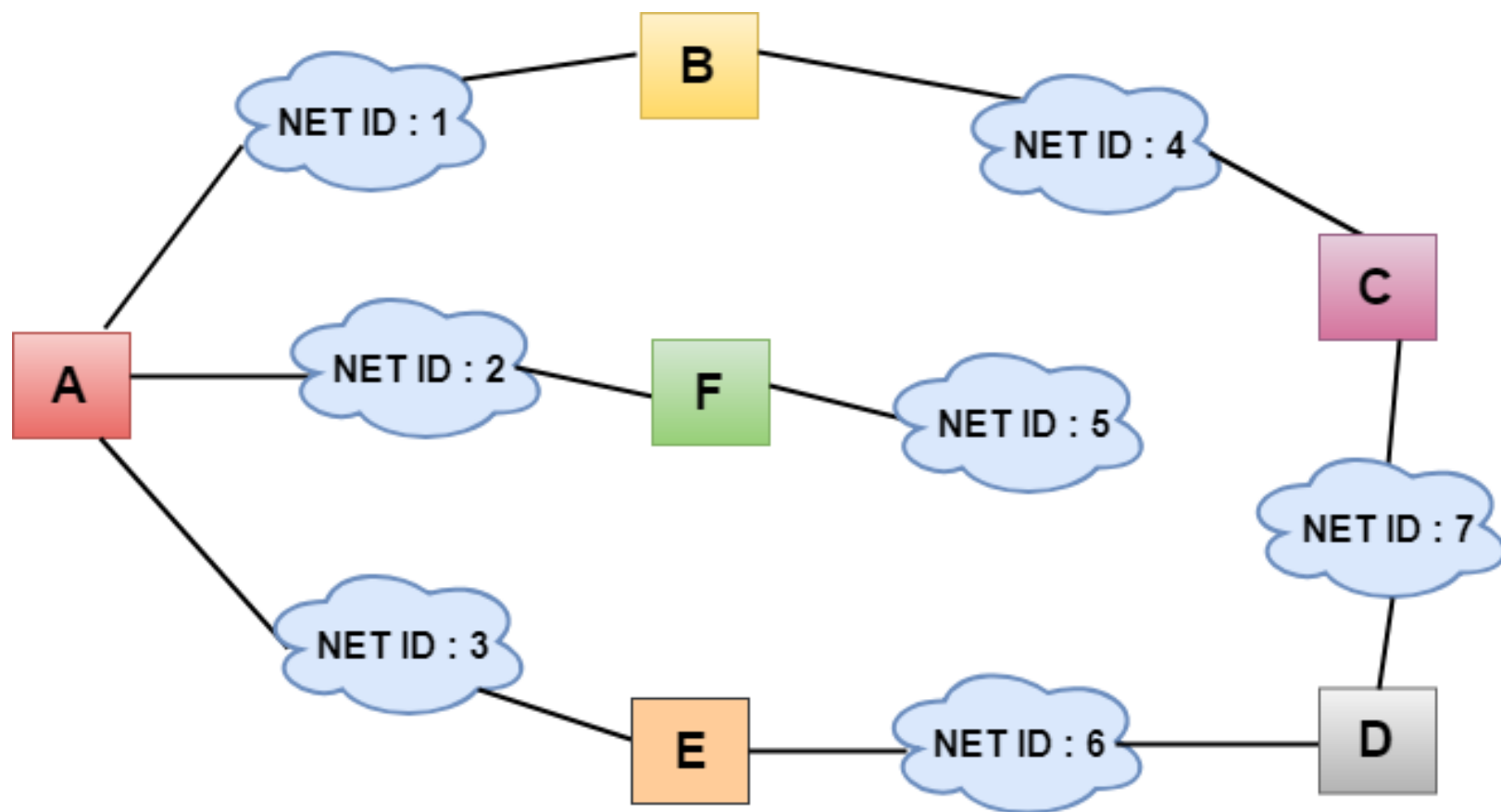
A route with a **smaller number of hops is considered as the best** route.

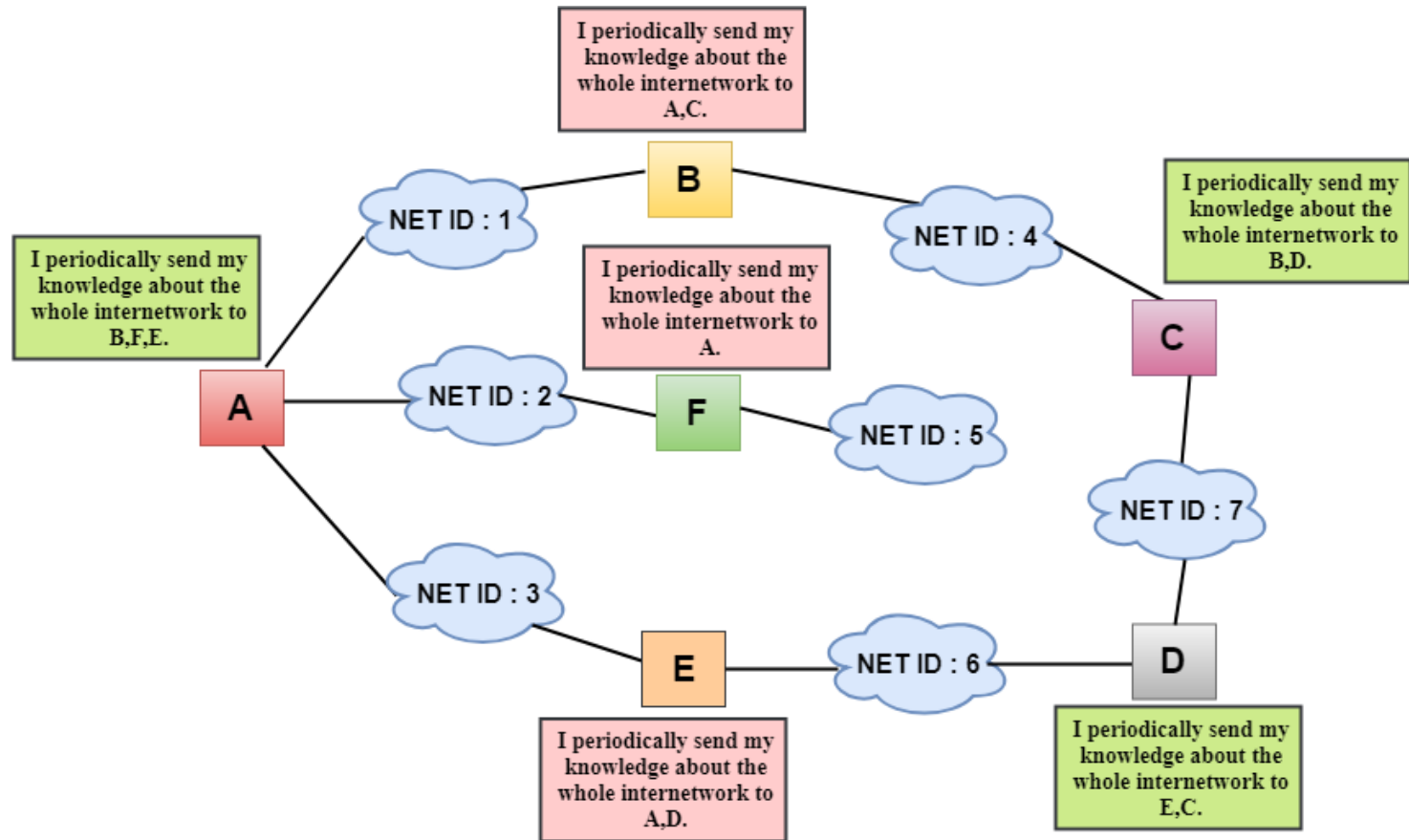
Every router **advertises its set best routes** to other routers.

Ultimately, all routers build up their network topology based on the advertisements of their peer routers.

For example, **Routing Information Protocol (RIP).**

Note: In Distance vector algorithm, node x update its table when it either see any cost change in one directly linked nodes or receives any vector update from some neighbor.



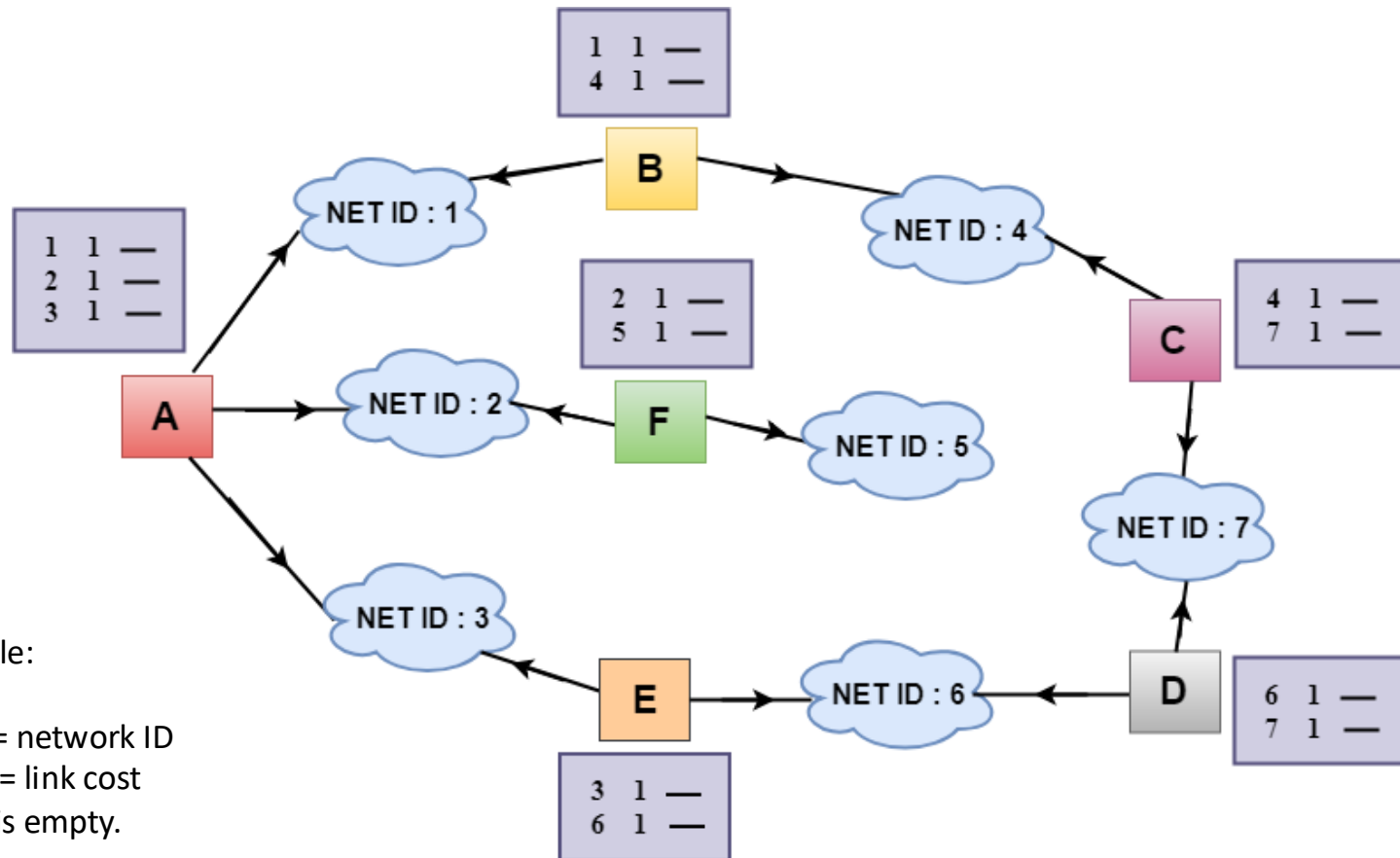


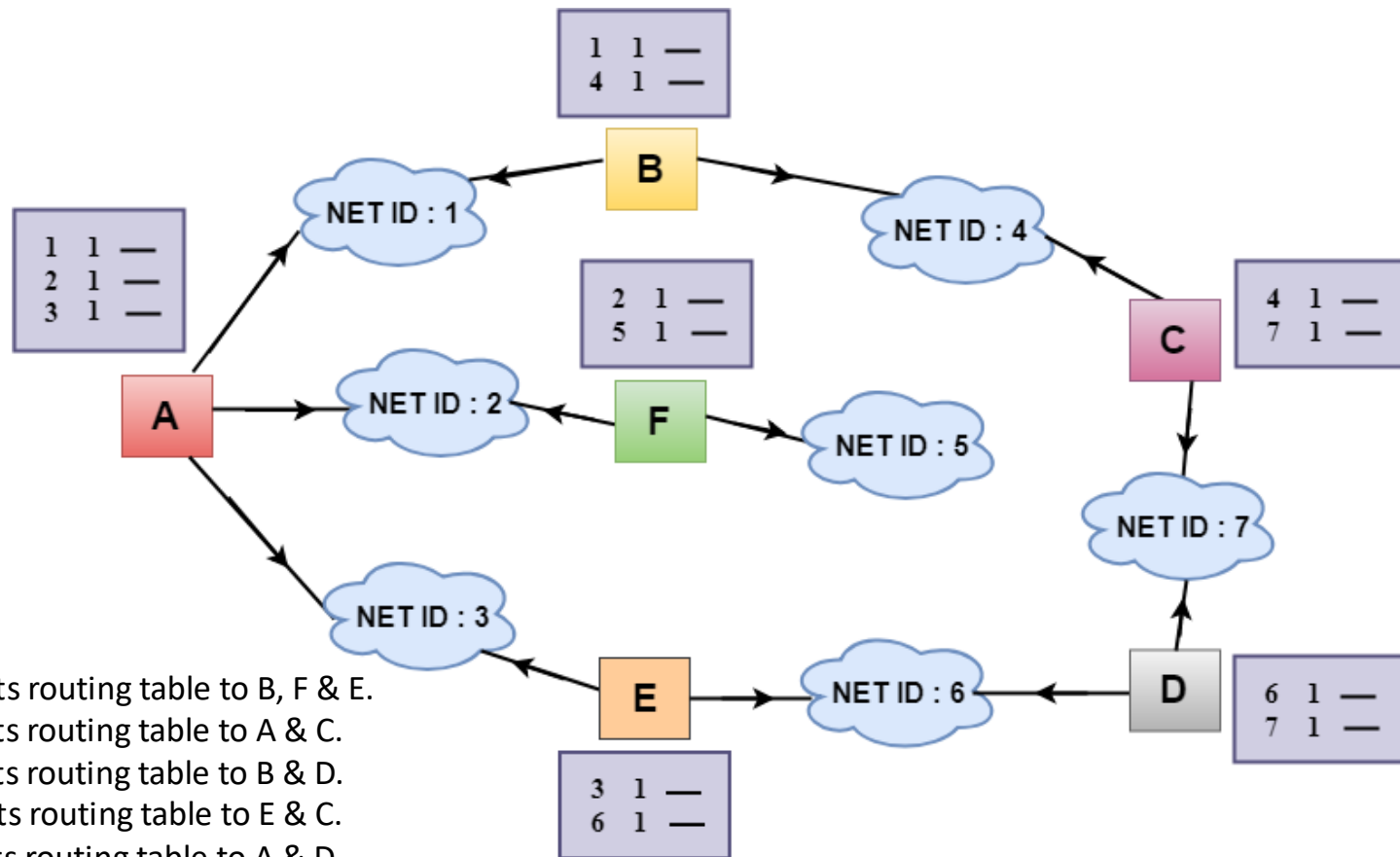
1. Router sends the knowledge to the immediate neighbors.
2. The neighbors add this knowledge to their own knowledge and sends the updated table to their own neighbors.
3. In this way, routers get its own information plus the new information about the neighbors.

4. Two process occurs:
Creating the Table
Updating the Table

NET ID	Cost	Next Hop
- - - - -	- - - - -	- - - - -
- - - - -	- - - - -	- - - - -
- - - - -	- - - - -	- - - - -
- - - - -	- - - - -	- - - - -
- - - - -	- - - - -	- - - - -

1. Creating the Table





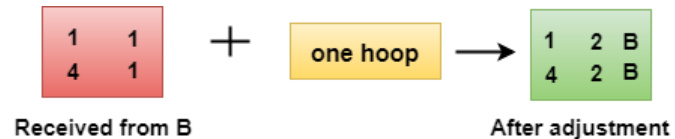
2. Updating the Table

A receives a routing table from B.

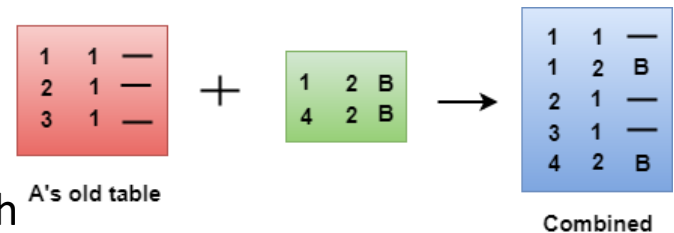
Routing table of B shows how packets can move to networks 1 & 4.

Packets from A to B can reach in one hop.

So, 1 is added to all costs in the B's table.



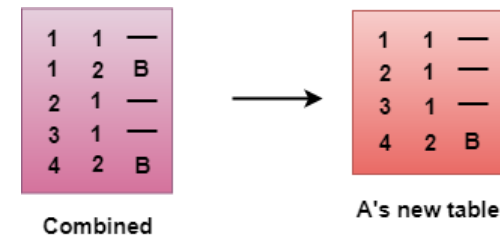
A then combines this table with its own table.



A removes the duplicate data & keeps the one with

A can send to network 1 > w/o any hop.

A can send via B. **[removed]**



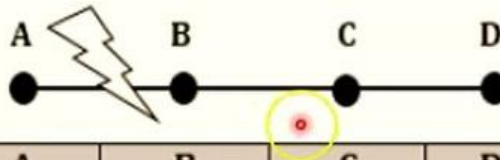
Disadvantage of Distance Vector Routing

The problem with distance vector routing is its **slowness in converging** to the correct answer. This is due to a problem called **count to infinity problem**.

COUNT TO INFINITY PROBLEM

- Counting to infinity is just another name for a **routing loop**.
- In distance vector routing, routing loops usually occur when an interface goes down.
- It can also occur when two routers send updates to each other at the same time.

Link Between A & B is Broken



	A	B	C	D
A	0, -	1, A	2, B	3, C
B	1, B	0, -	2, C	3, D
C	2, B	1, C	0, -	1, C
D	3, B	2, C	1, D	0, -

- Once again, routers exchange their tables.
- When C receives B's routing table, it sees that B has changed the weight of its link to A from 1 to 3, so C updates its table and changes the weight of the link to A to 4 (1 for C to B, and 3 for B to A, as B said).
- This process loops until all nodes find out that the weight of link to A is infinity.
- This situation is shown in the table below.
- In this way, Distance Vector Algorithms have a slow convergence rate.

	B	C	D
Sum of Weight to A after link cut	∞ , A	2, B	3, C
Sum of Weight to A after 1 st updating	3, C	2, B	3, C
Sum of Weight to A after 2 nd updating	3, C	4, B	3, C
Sum of Weight to A after 3 rd updating	5, C	4, B	5, C
Sum of Weight to A after 4 th updating	5, C	6, B	5, C
Sum of Weight to A after 5 th updating	7, C	6, B	7, C
Sum of Weight to A after n th updating			
∞	∞	∞	∞

- One way to solve this problem is for routers to **send information only to the neighbors** that are not exclusive links to the destination.
- For example, in this case, **C shouldn't send any information to B about A**, because B is the only way to A.

Link State Routing Protocol

Two Phases

Reliable Flooding

Initial state: Each node knows the cost of its neighbors.

Final state: Each node knows the entire graph.

Route Calculation

Each node uses Dijkstra's algorithm on the graph to calculate the optimal routes to all nodes.

Important Notations

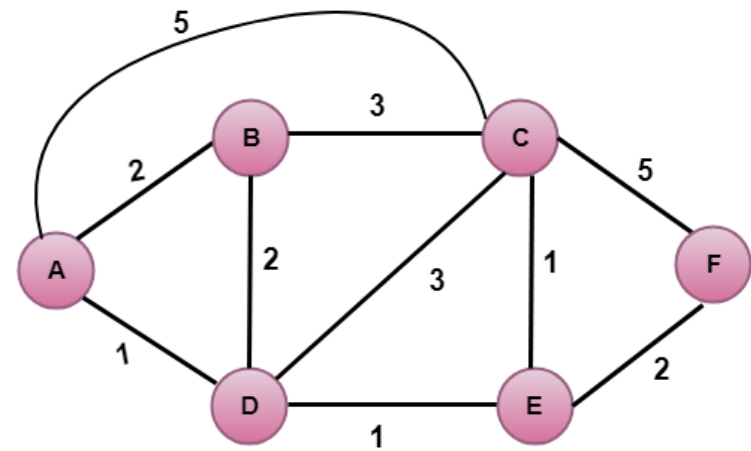
- $c(i, j)$: Link cost from node i to node j .

If i and j nodes are not directly linked, then $c(i, j) = \infty$.

- $D(v)$: Link cost from source to destination v that has the least cost currently.

- $P(v)$: Previous node (neighbor of v) in the least cost path.

- N : Total number of nodes in the network.



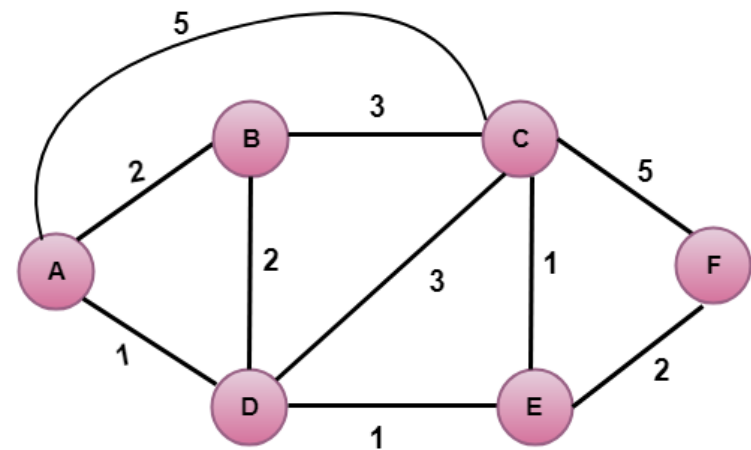
Step	N	D(B),P(B)	D(C),P(C)	D(D),P(D)	D(E),P(E)	D(F),P(F)
1	A	2,A	5,A	1,A	∞	∞

Source vertex is A.

Vertex D contains the least cost path.

D is added in N.

We need to determine a least-cost path through D vertex, now.



Step	N	D(B),P(B)	D(C),P(C)	D(D),P(D)	D(E),P(E)	D(F),P(F)
1	A	2,A	5,A	1,A	∞	∞
2	AD	2,A	4,D		2,D	∞

Calculating shortest path from A to B

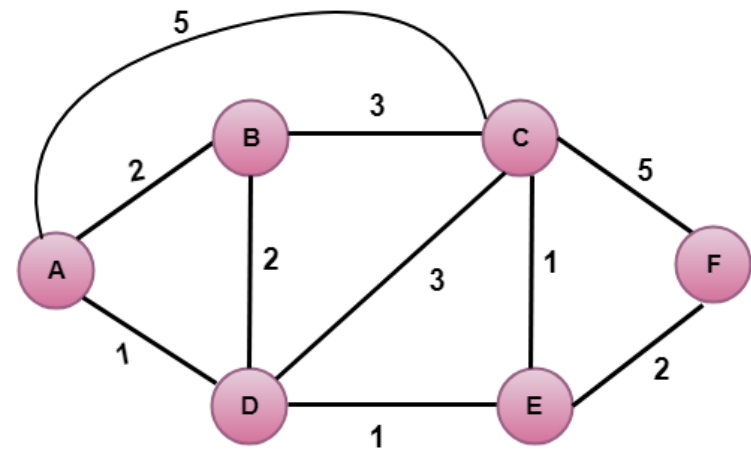
$$D(B) = \min(D(B) , D(D) + c(D,B)) = \min(2, 1+2) = \min(2, 3) = 2$$

Calculating shortest path from A to C

$$D(C) = \min(D(C) , D(D) + c(D,C)) = \min(5, 1+3) = \min(5, 4) = 4$$

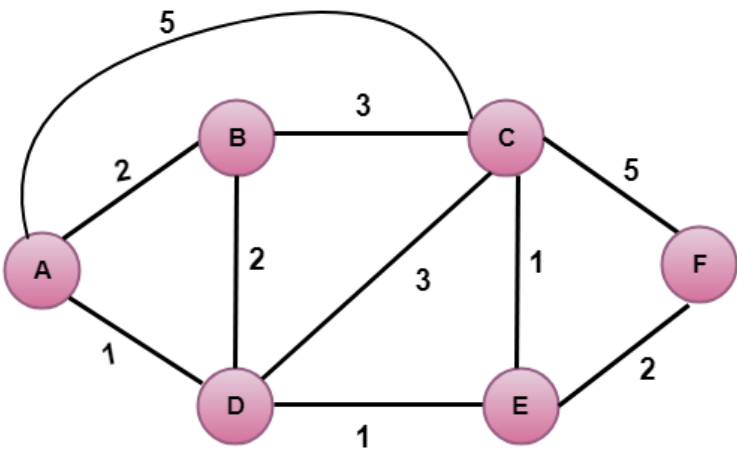
Calculating shortest path from A to E

$$D(E) = \min(D(E) , D(D) + c(D,E)) = \min(\infty, 1+1) = \min(\infty, 2) = 2$$



Step	N	D(B),P(B)	D(C),P(C)	D(D),P(D)	D(E),P(E)	D(F),P(F)
1	A	2,A	5,A	1,A	∞	∞
2	AD	2,A	4,D		2,D	∞

*Vertex B & E contain the least cost path.
Let's consider the next vertex as E.*



Step	N	D(B),P(B)	D(C),P(C)	D(D),P(D)	D(E),P(E)	D(F),P(F)
1	A	2,A	5,A	1,A	∞	∞
2	AD	2,A	4,D		2,D	∞
3	ADE	2,A	3,E			4,E

Calculating shortest path from A to B

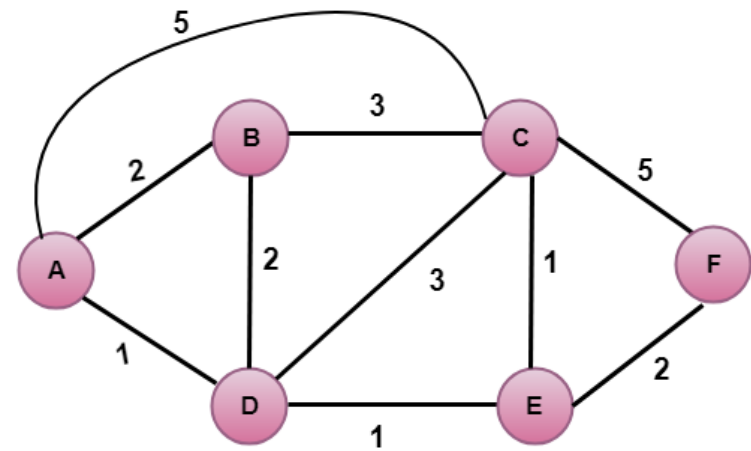
$$D(B) = \min(D(B) , D(E) + c(E,B)) = \min(\textcolor{red}{2}, \textcolor{red}{2} + \infty) = \min(\textcolor{red}{2}, \infty) = 2$$

Calculating shortest path from A to C

$$D(C) = \min(D(C) , D(E) + c(E,C)) = \min(\textcolor{red}{4}, \textcolor{red}{2} + \textcolor{red}{1}) = \min(\textcolor{red}{4}, \textcolor{red}{3}) = 3$$

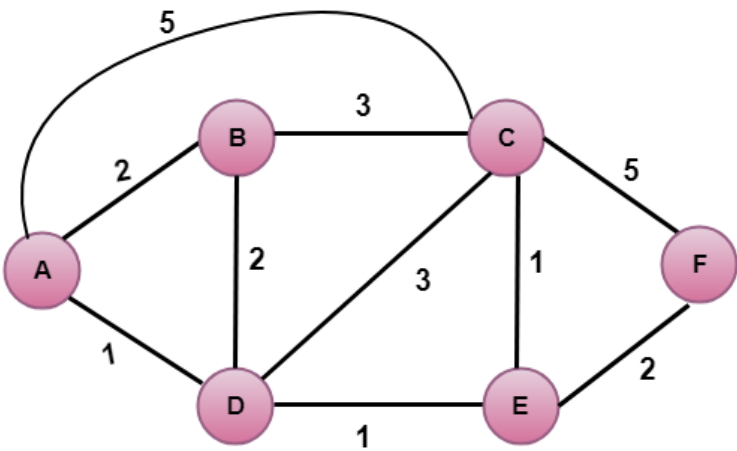
Calculating shortest path from A to F

$$D(F) = \min(D(F) , D(E) + c(E,F)) = \min(\infty, \textcolor{red}{2} + \textcolor{red}{2}) = \min(\infty, \textcolor{red}{4}) = 4$$



Step	N	D(B),P(B)	D(C),P(C)	D(D),P(D)	D(E),P(E)	D(F),P(F)
1	A	2,A	5,A	1,A	∞	∞
2	AD	2,A	4,D		2,D	∞
3	ADE	2,A	3,E			4,E

*Vertex B contains the least cost path.
Let's consider the next vertex as B.*



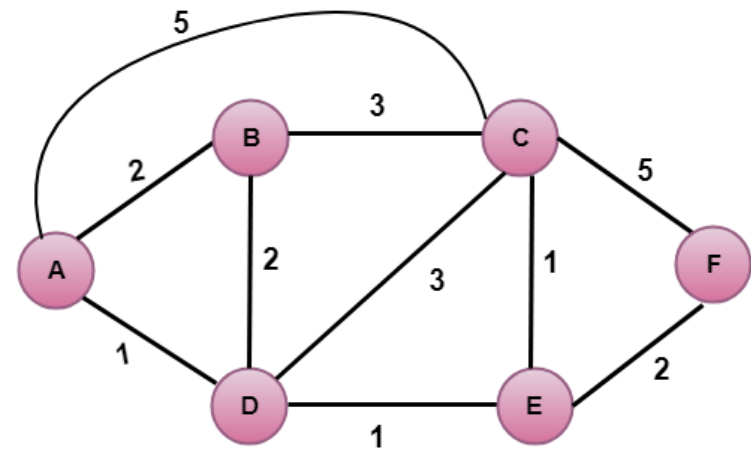
Step	N	D(B),P(B)	D(C),P(C)	D(D),P(D)	D(E),P(E)	D(F),P(F)
1	A	2,A	5,A	1,A	∞	∞
2	AD	2,A	4,D		2,D	∞
3	ADE	2,A	3,E			4,E
4	ADEB		3,E			4,E

Calculating shortest path from A to C

$$D(C) = \min(D(C) , D(B) + c(B,C)) = \min(3, 2 + 3) = \min(3, 5) = 3$$

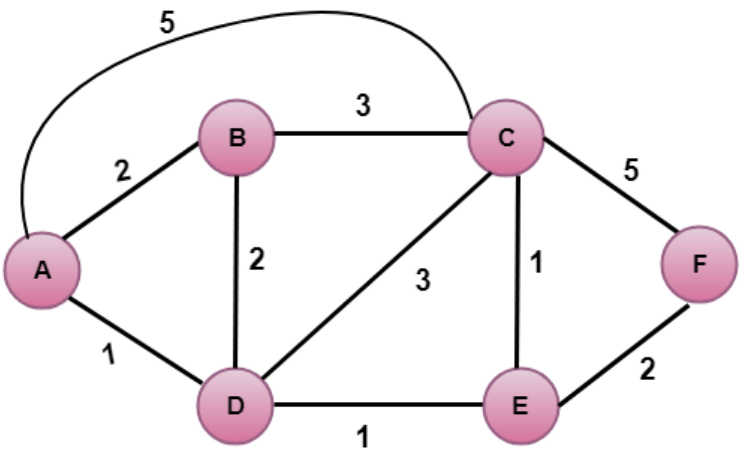
Calculating shortest path from A to F

$$D(F) = \min(D(F) , D(B) + c(B,F)) = \min(4, 2 + \infty) = \min(4, \infty) = 4$$



Step	N	D(B),P(B)	D(C),P(C)	D(D),P(D)	D(E),P(E)	D(F),P(F)
1	A	2,A	5,A	1,A	∞	∞
2	AD	2,A	4,D		2,D	∞
3	ADE	2,A	3,E			4,E
4	ADEB		3,E			4,E

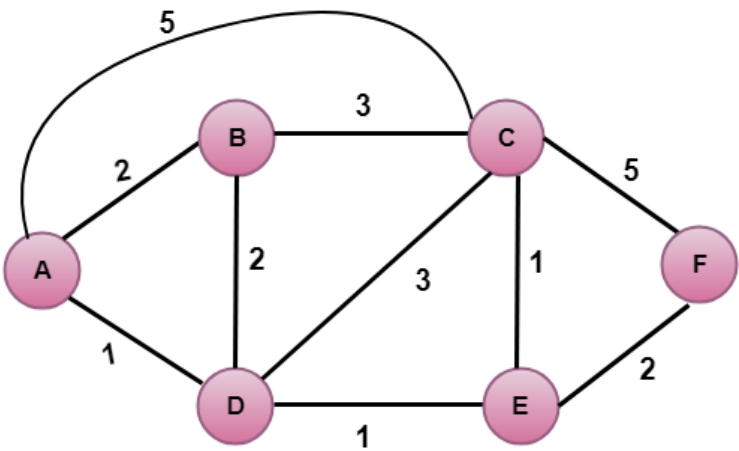
*Vertex C contains the least cost path.
Let's consider the next vertex as C.*



Step	N	D(B),P(B)	D(C),P(C)	D(D),P(D)	D(E),P(E)	D(F),P(F)
1	A	2,A	5,A	1,A	∞	∞
2	AD	2,A	4,D		2,D	∞
3	ADE	2,A	3,E			4,E
4	ADEB		3,E			4,E
5	ADEBC					4,E

Calculating shortest path from A to F

$$D(F) = \min(D(F), D(C) + c(C,F)) = \min(4, 3 + 5) = \min(4, 8) = 4$$



Step	N	D(B),P(B)	D(C),P(C)	D(D),P(D)	D(E),P(E)	D(F),P(F)
1	A	2,A	5,A	1,A	∞	∞
2	AD	2,A	4,D		2,D	∞
3	ADE	2,A	3,E			4,E
4	ADEB		3,E			4,E
5	ADEBC					4,E
6	ADEBCF					

Vertex F is the final node in the least cost path.

Congestion Control techniques

Congestion Control Techniques

```
graph TD; A[Congestion Control Techniques] --> B[Open loop Congestion Control]; A --> C[Closed loop Congestion Control];
```

Open loop Congestion Control

1. Policies are applied to prevent congestion before it happens.
2. Congestion control is handled either by the source or the destination.

Closed loop Congestion Control

Policies are used to treat or alleviate congestion after it happens.

Open Loop Congestion Control

1. Retransmission Policy

- Retransmission may increase congestion. To prevent congestion, retransmission timers designed.

2. Window Policy

- Selective repeat window should be adopted to reduce duplication while retransmitting lost packet.

3. Discarding Policy

- Less sensitive/corrupted audio packets discarded while also maintaining the quality.

4. Acknowledgment Policy

- Acknowledgement sent for N packets rather than sending for a single packet.

5. Admission Policy

- Switches check for the resource requirement of a network flow before transmitting the data further.

Closed Loop Congestion Control

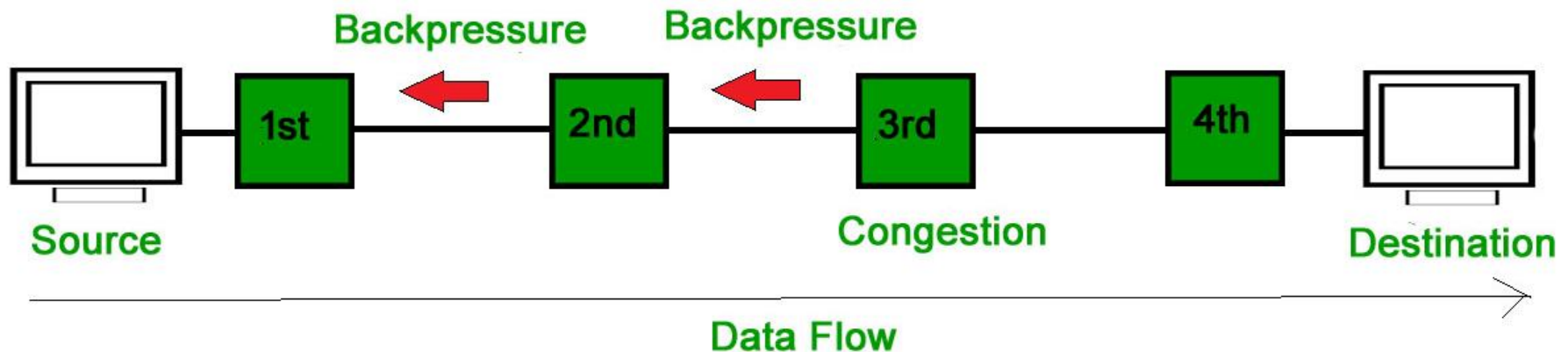
1. Backpressure :

Only to virtual circuit.

Congested node stops receiving packets from upstream node.

Upstream node(s) become congested.

Backpressure propagate in the opposite direction of data flow.

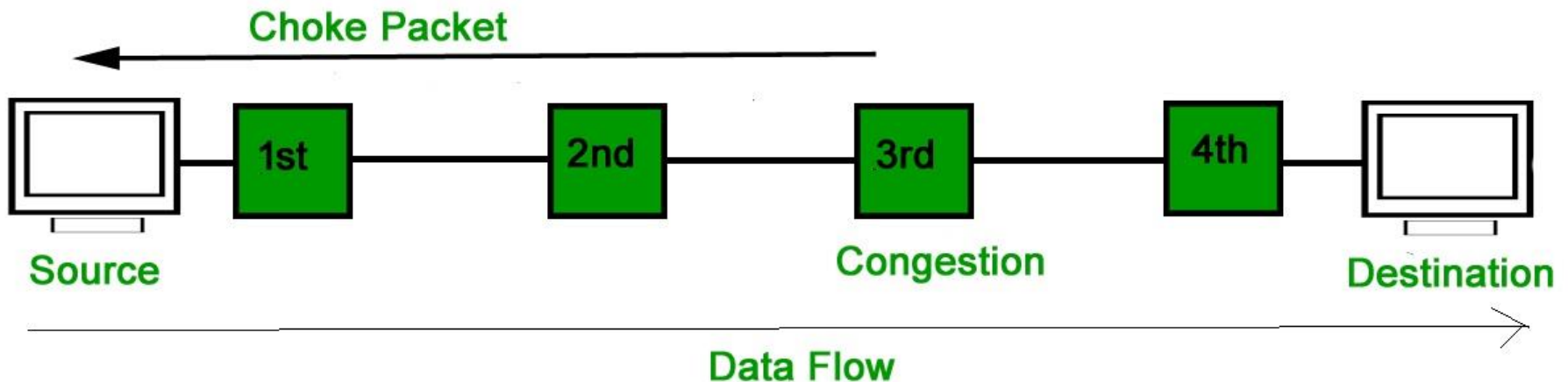


Closed Loop Congestion Control

2. Choke Packet Technique :

Both virtual networks & datagram subnets.

When resource utilization $>$ threshold value, router **directly** sends a choke packet(feedback) to the source telling it to reduce the traffic.



Closed Loop Congestion Control

3. Implicit Signalling :

No communication between congested nodes & the source.

Source guesses that there is congestion in a network.

Example - sender sends several packets & there is no acknowledgment for a while.

Closed Loop Congestion Control

4. Explicit Signalling :

Congested node explicitly sends a packet to the source or destination to inform about congestion.

- **Forward Signaling** : Destination is warned about congestion.
- **Backward Signaling** : Source is warned about congestion.
- **Difference b/w choke packet & explicit signaling :**

Explicit signal = included in the packets that carry data

Choke packet technique= a different packet included

Congestion control in data-gram and sub-nets

Congestion Control approaches used in datagram Subnet

A. Choke Packets

Drawback –

The problem with choke packet technique is that the action to be taken by the source host on receiving a choke packet is voluntary and not compulsory.

1. Used in virtual circuits as well as in datagram sub-nets.
2. Each router associates a real variable with each of its output line.
 - Value of real variable = $(0,1)$
 - Indicates percentage utilization of output line.
 - Value > Threshold value = Output line enters the warning state.
3. If newly arrived packet = from an output line in warning state >>> Router will send back the choke packets.
4. Depending upon the threshold value, the choke packets can contain a mild warning a stern warning, or an ultimatum.

B. Load Shedding

1. PRINCIPLE: If router is inundated by the packets it cannot handle, packets would be thrown away.
2. Packet dropped based on the type of the packet.
 - For File Transfer: Old packet is more important than the new packet. [policy for file transfer is called wine (old is better than new)]
 - For multimedia: New packet is more important than the old packet. [policy for file transfer is called milk (new is better than old)]
3. To implement such an intelligent discard policy, co-operation from the sender is essential.
4. The application should mark their packets in priority classes to indicate how important they are.
5. When the packets are to be discarded,
 - Routers can first drop packets from the lowest class (i.e., the packets which are least important).
 - Then routers will discard the packets from next lower class and so on.

C. Jitter control

1. Jitter = Variation in delay for the packet belonging to the same flow.

File Transfer can tolerate jitter.

Real time audio/video cannot tolerate jitter.

2. When a packet arrives at a router, the router will check to see whether the packet is behind or ahead and by what time. [This information is stored in the packet and updated at every hop]

If packet is ahead of the schedule = then router will hold it for a slight longer time

If packet is behind schedule = then router will try to send it out as quickly as possible.

This will help in keeping the average delay per packet constant and will avoid time jitter.

Traffic Shaping Algorithms –

1. Leaky Bucket
2. Token Bucket

Traffic shaping helps

1. to regulate rate of data transmission &
2. reduces congestion.

1. Leaky Bucket

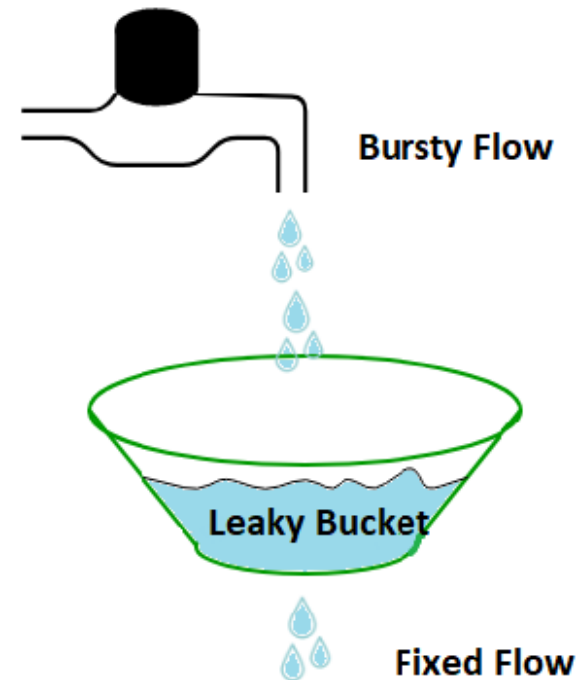
1. In a bucket, we are pouring water in a random order.

2. We want the flow at fixed rate.

3. Make a hole at the bottom of the bucket

4. Water will come out at fixed rate

5. If the bucket is full, we stop pouring water in it.



Leaky Bucket (contd.)

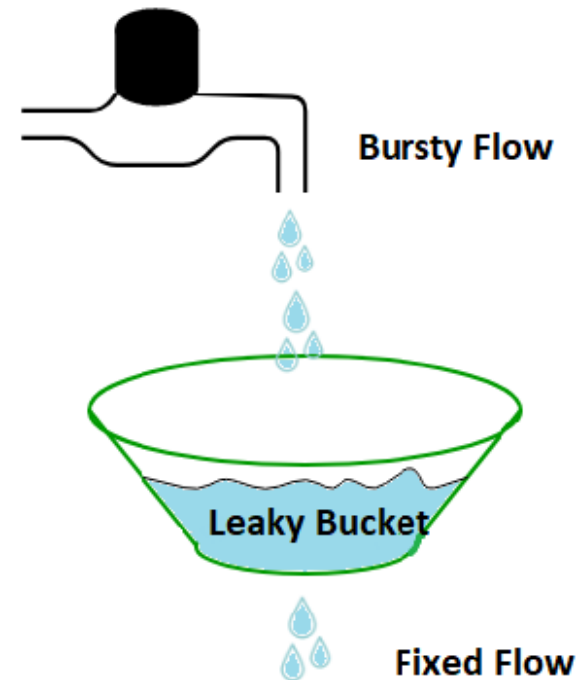
1. Input rate can vary.

2. We want the flow at fixed rate.

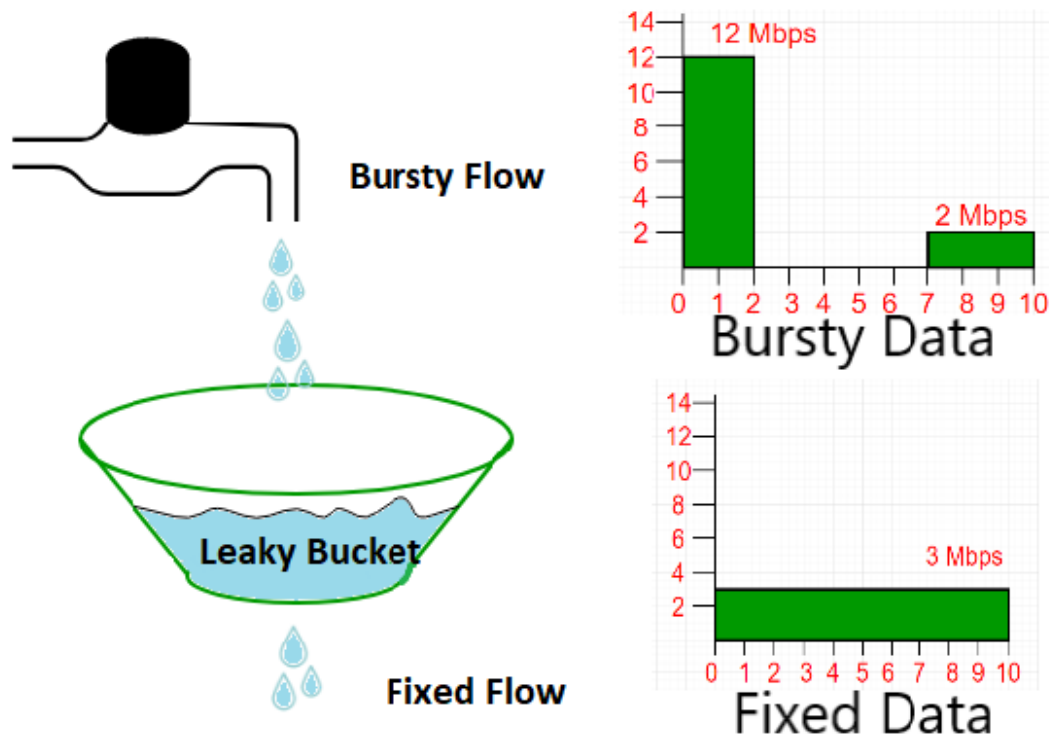
3. Bursty flow chunks are stored in the bucket and sent out at an average rate.

4. Output rate remains constant.

5. If the bucket is full, we stop pouring water in it.

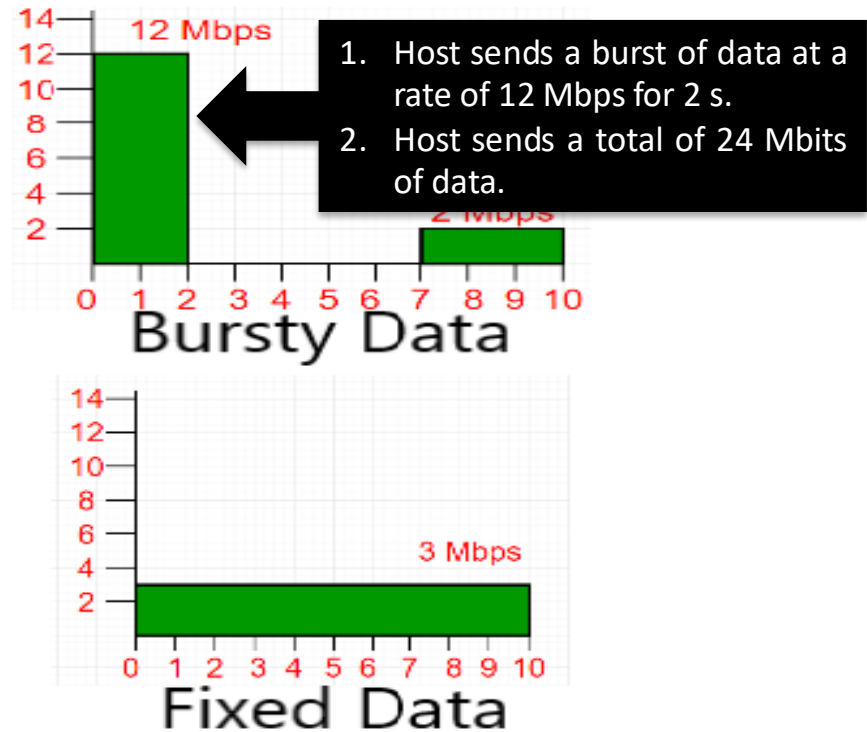


Leaky Bucket (contd.)



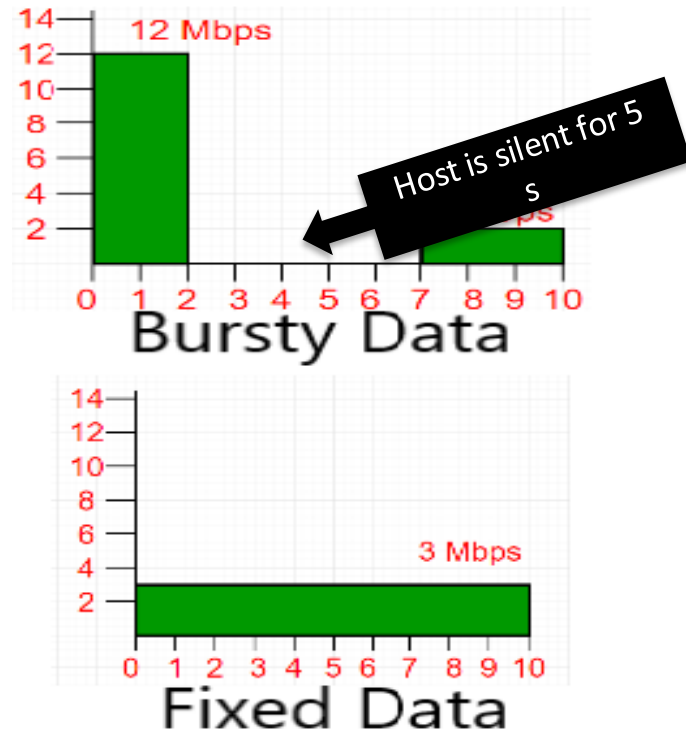
Leaky Bucket (contd.)

1. Network has committed a bandwidth of 3 Mbps for a host.
2. The use of the leaky bucket shapes the input traffic to make it conform to this commitment.



Leaky Bucket (contd.)

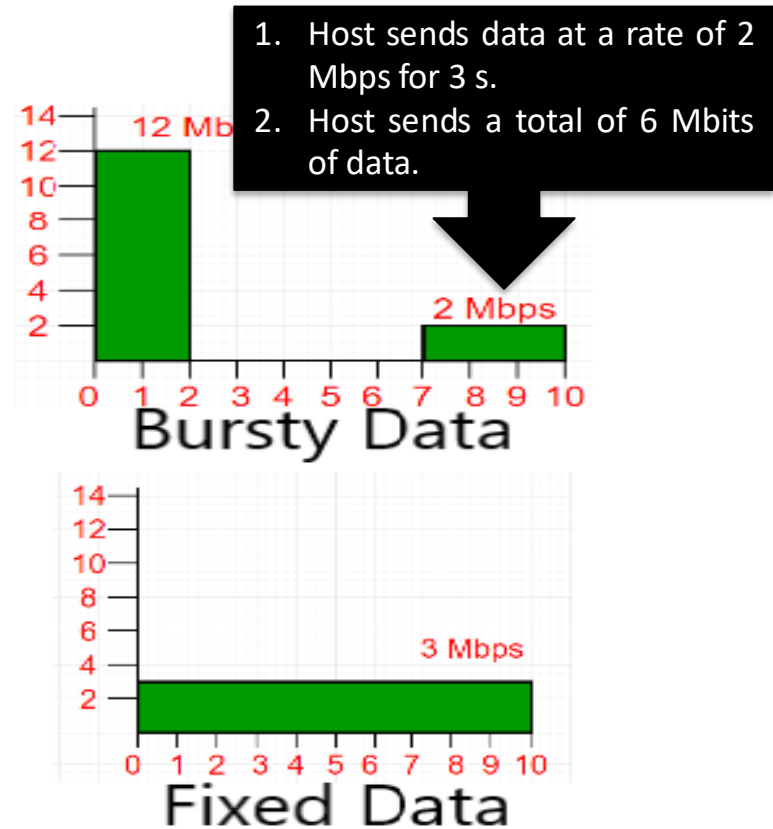
1. Network has committed a bandwidth of 3 Mbps for a host.
2. The use of the leaky bucket shapes the input traffic to make it conform to this commitment.



Leaky Bucket (contd.)

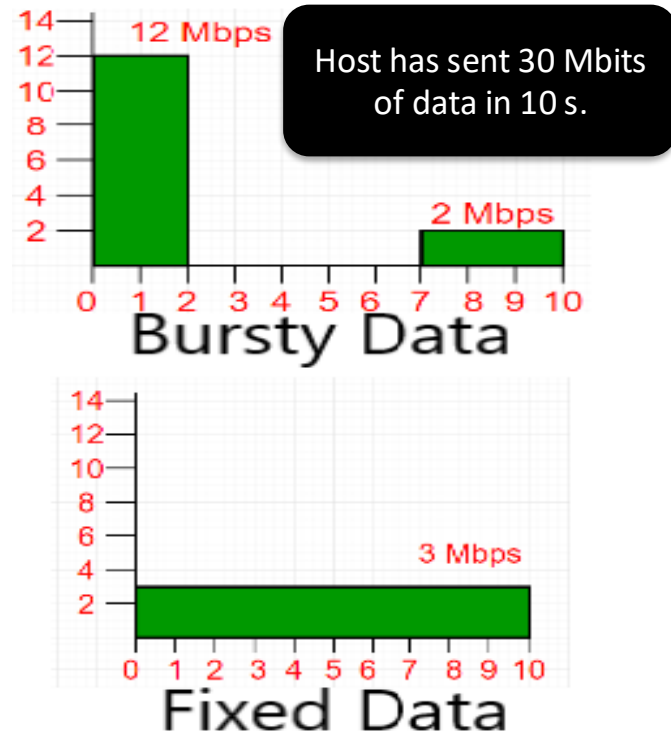
1. Network has committed a bandwidth of 3 Mbps for a host.

2. The use of the leaky bucket shapes the input traffic to make it conform to this commitment.



Leaky Bucket (contd.)

1. Network has committed a bandwidth of 3 Mbps for a host.
2. The use of the leaky bucket shapes the input traffic to make it conform to this commitment.

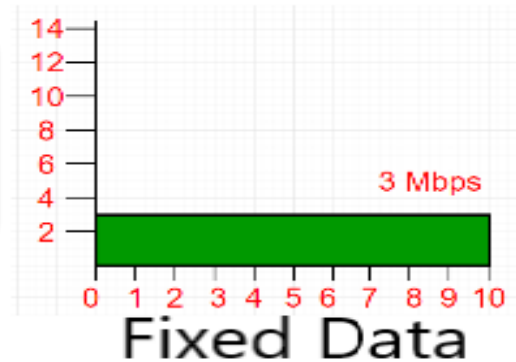
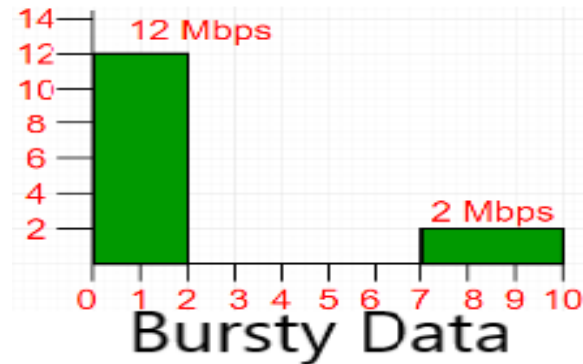


Leaky Bucket (contd.)

1. Network has committed a bandwidth of 3 Mbps for a host.

2. The use of the leaky bucket shapes the input traffic to make it conform to this commitment.

The leaky bucket smooths the traffic by sending out data at a rate of 3 Mbps during the same 10 s.



2. Token Bucket

Need of Token Bucket Algorithm

The leaky bucket algorithm enforces output patterns at the average rate, no matter how busy the traffic is.

So, to deal with the more traffic, we need a flexible algorithm so that the data is not lost.

One such approach is the token bucket algorithm.

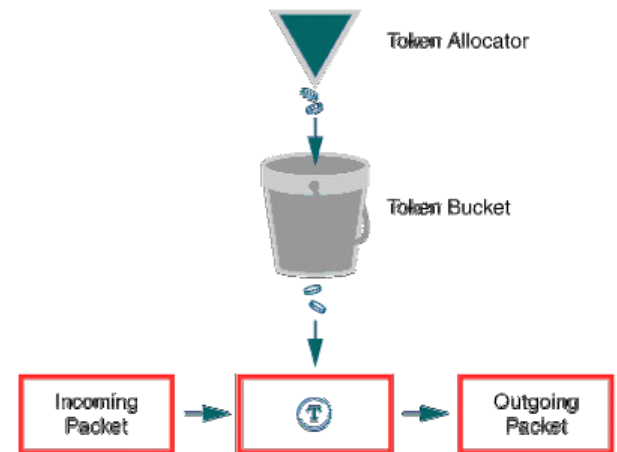
Token Bucket Algorithm

In regular intervals tokens are thrown into the bucket.

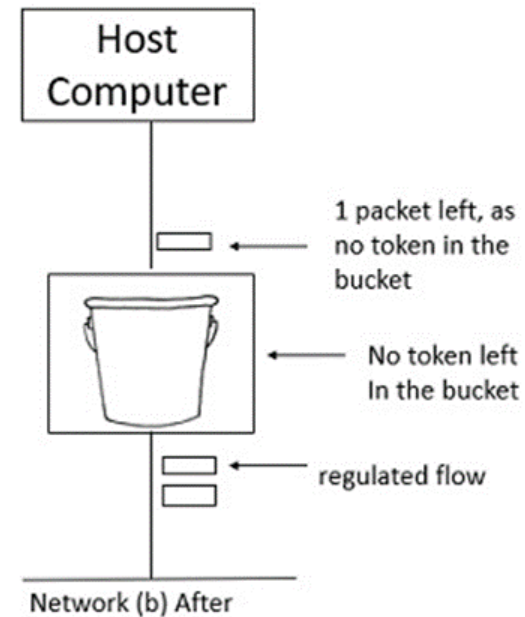
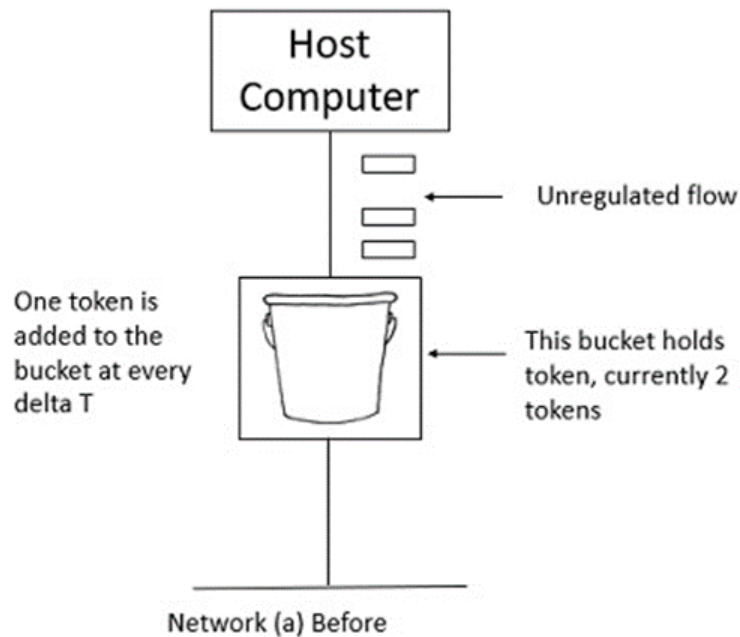
The bucket has a maximum capacity f .

If the packet is ready, then a token is removed from the bucket, and the packet is sent.

If there is no token in the bucket, the packet cannot be sent.



Token Bucket



Difference between Leaky Bucket & Token Bucket

Token bucket algorithm is less restrictive that means it allows more traffic.

The limit of busyness is restricted by the number of tokens available in the bucket at a particular instant of time.

A variable is used to count the tokens.

For every t seconds, counter is incremented.

Whenever a packet is sent, the counter is decremented.

When the counter reaches zero, no further packet is sent out.